WBS TRAINING

Observer Pattern

GoF Entwurfsmuster

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

WBS TRAINING

Inhalt

1 Observer Pattern / Beobachtermuster 2

1.1 Einsatzzweck 2

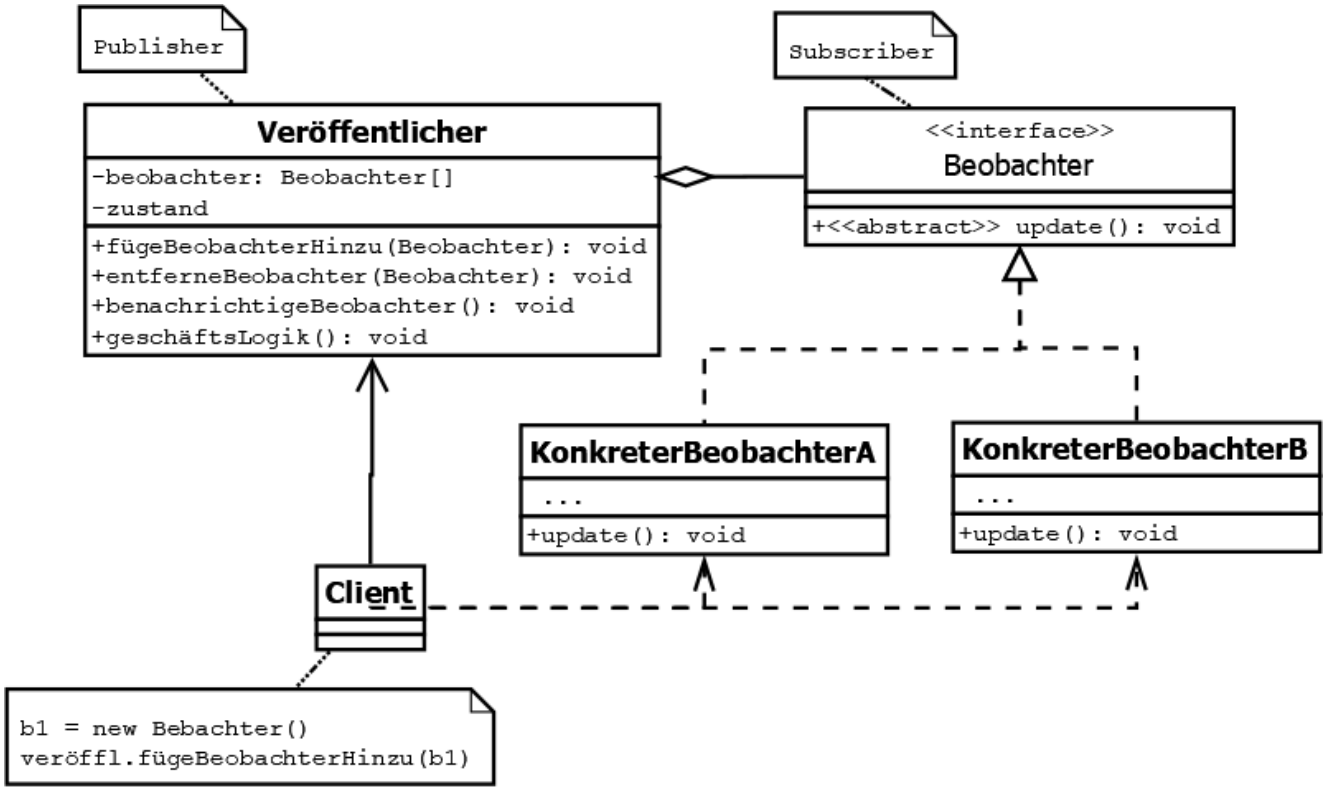
1.2 Informationen 3

1.3 Struktur Beobachtermuster 3

1.4 Implementierungsschritte Beobachtermuster 4

1 Observer Pattern / Beobachtermuster

Kategorie: Verhaltensmuster



1.1 Einsatzzweck

Das Problem erläutert an einem kleinen Beispiel:
Stellen Sie sich vor es gibt zwei Objekte ein Sportfachgeschäft und eine Kundin, die gerne das neueste Trikot ihres Lieblingsvereins ergattern möchte, das bald erhältlich sein soll.

Es gibt nun zwei Strategien, wie die Kundin an ihr begehrtes Stück kommen könnte:

1. Sie geht täglich in das Sportfachgeschäft und schaut, ob das Trikot nun erhältlich ist. Der Nachteil ist, dass die meisten ihrer Wege überflüssig sind.

2. Das Sportfachgeschäft versendet immer Unmengen von Emails an alle ihre Kunden, sobald ein neues Produkt erhältlich ist. Viele Kunden könnten das aber auch als Spam auffassen.

Wir haben also folgenden Konflikt: Entweder verschwendet die Kundin ihre Zeit, um die Produktverfügbarkeit zu prüfen oder das Sportfachgeschäft verschwendet Ressourcen, um die falschen Kunden zu benachrichtigen und zu verärgern. Lösen könnten wir diesen Konflikt mit Hilfe des Beobachter-Musters.

1.2 Informationen

- Das Beobachter-Muster ist ein Verhaltensmuster, ist auch bekannt als Event-Subscriber oder Listener-Muster.

- Wir definieren ein Abonnement Mechanismus, um mehrere Objekte zu benachrichtigen, wenn mit dem beobachteten Objekt besondere Ereignisse geschehen.

- Das Objekt, dass die interessanten Zustände hat benachrichtigt andere Objekte über die Änderungen seines Zustands. Darum nennen wir dieses Objekt auch Publisher (Veröffentlicher). Die anderen Objekte, die diese Änderungen interessiert verfolgen, werden Subscribers (Abonnenten) genannt.

- Das Observer Pattern verlangt, dass ein Abonnement Mechanismus in der Veröffentlicher Klasse implementiert wird, so dass die interessierten Objekte den Strom an Events abonnieren oder deabonnieren können.

1.3 Struktur Beobachtermuster

Dieses Muster ist leichter zu implementieren, als man zuerst glauben mag.
Der Mechanismus besteht aus:

1. einem Array oder Listen Attribut, um eine Liste von Referenzen zu den Abonnenten zu speichern.

2. Verschiedene öffentliche Methoden, um die Abonnenten der Liste hinzuzufügen oder um diese wieder entfernen zu können.

Wann immer nun ein wichtiges Ereignis mit dem Veröffentlicher Objekt geschieht, wird dieses an seine Abonnenten weitergereicht und ruft die speziellen Benachrichtigungsmethoden auf die Objekte auf.

Echte Anwendungen haben häufig dutzende von verschiedenen Abonnementen Klassen, die Interesse an den Events derselben Veröffentlicher Klasse haben. Du möchtest den Veröffentlicher nicht mit all diesen Klassen eng verkuppeln, sondern dem Veröffentlicher sollte es egal sein, wer ihn abonniert. Darum ist es wichtig, dass alle Abonnement Klassen dasselbe Interface implementieren und dass der Veröffentlicher ausschließlich über dieses Interface mit ihnen kommuniziert.

Wir erreichen eine sehr lose Kopplung. Das Interface deklariert die Benachrichtigungsmethode mit einigen Parametern, die der Veröffentlicher verwenden kann, um kontextuelle Daten gemeinsam mit seinen Benachrichtigungen weiterzugeben.

Analogie in der echten Welt: Abodienste sorgen dafür, dass Kunden nicht mehr in ein Geschäft gehen müssen, um zu sehen, ob das Objekt der Begierde (z.B. ein technisches Magazin) vorrätig ist. Stattdessen versendet der Veröffentlicher Hinweise an die Abonnenten. Der Veröffentlicher hat eine Liste vor mit all seinen Abonnenten und weiß an welchen Magazinen sie jeweils Interesse haben. Die Abonnenten können jederzeit das Abo abbestellen und sich aus der Liste entfernen lassen.

1.4 Implementierungsschritte Beobachtermuster

1. Erstelle ein allgemeines Beobachter Interface. In den meisten Fällen besteht dies aus einer einzigen update() Methode. Die Methode kann über verschiedene Parameter verfügen, wodurch es dem Publisher ermöglicht wird, Ereignisdetails mitzuliefern. Dadurch ist der Publisher nicht mit konkreten Klassen verkuppelt, sondern kennt nur den abstrakten Typ.

2. Alle konkreten Beobachter Klassen implementieren das Interface und überschreiben jeweils die abstrakte update() Methode auf ihre eigene Weise.

3. Erstelle die Publisher Klasse. Die Publisher Klasse enthält ein Listen oder Array Attribut vom Typ des Beobachter Interfaces. Der Publisher braucht außerdem öffentliche Methoden um Beobachter/Abonnenten der Liste bzw. dem Array hinzuzufügen und wieder aus der Liste zu entfernen. Dem Publisher implementieren wir außerdem eine Methode um die Beobachter zu benachrichtigen, wenn es interessante Änderungen gibt. Mit der Benachrichtigungsmethode wird der Publisher über seine Liste iterieren und für alle interessierten Beobachter deren update() Methode aufrufen.

4. Der Client kreiert Publisher und Abonnenten Objekte und registriert die Abonnenten für die Publisher Updates.